# System architecture

Proposed service boundaries and the path from scientific sources to the browser.

PROPOSED • Not implemented • Scientific goal, scoring weights and hosting provider remain undecided

Browser → imagery / tile CDN • Batch sources → processing jobs → storage + database

01

## Browser application

React + TypeScript + CesiumJS proposed. Global discovery leads into a detailed Jezero Marswalk. Local layers, science stops, round-trip planning and a field view are central. Scientific goal remains null until selected.

02

## Static hosting / CDN

Serve compiled frontend assets through HTTPS. Browser requests imagery and 3D tiles directly from approved providers or object storage. Keep bulk tile traffic away from the API process.

03

## Application API

Python + FastAPI proposed. Validate requests, search the catalog, return evidence, apply mission constraints and calculate lightweight rankings. Publish a versioned OpenAPI contract.

04

## Spatial database

PostgreSQL + PostGIS proposed. Store crater records, footprints, scientific observations, dataset versions and analysis results. Use spatial indexes and explicit Mars coordinate reference systems.

05

## Processing jobs

Python + GDAL proposed. Import source data and precompute regional terrain features. Use a separate batch process for heavy raster work; persistent queued jobs can be added when interactive routing needs them.

06

## Scientific assets

Versioned object storage for raw snapshots, derived rasters and display tiles. NASA / USGS / PDS are source systems, not databases we assume are always reachable. Cesium ion is an optional visual terrain provider.

Read technical-brief.md for linked sources, assumptions and competition timing.